Colecţii

Programare Orientată pe Obiecte

Colecţii

- Ce sunt colecţiile ?
- Interfeţe ce descriu colecţii
- Implementări ale colecţiilor
- Folosirea eficientă a colecţiilor
- Algoritmi polimorfici
- Tipuri generice
- Iteratori şi enumerări

Ce sunt colecţiile ?

- O colecţie este un obiect care grupează mai multe elemente într-o singură unitate.
- Tipul de date al elementelor dintr-o colecţie este Object.

Tipuri de date reprezentate:

- vectori
- liste înlănţuite
- stive
- mulţimi matematice
- tabele de dispersie
- dicţionare, etc.

Arhitectura colecţiilor

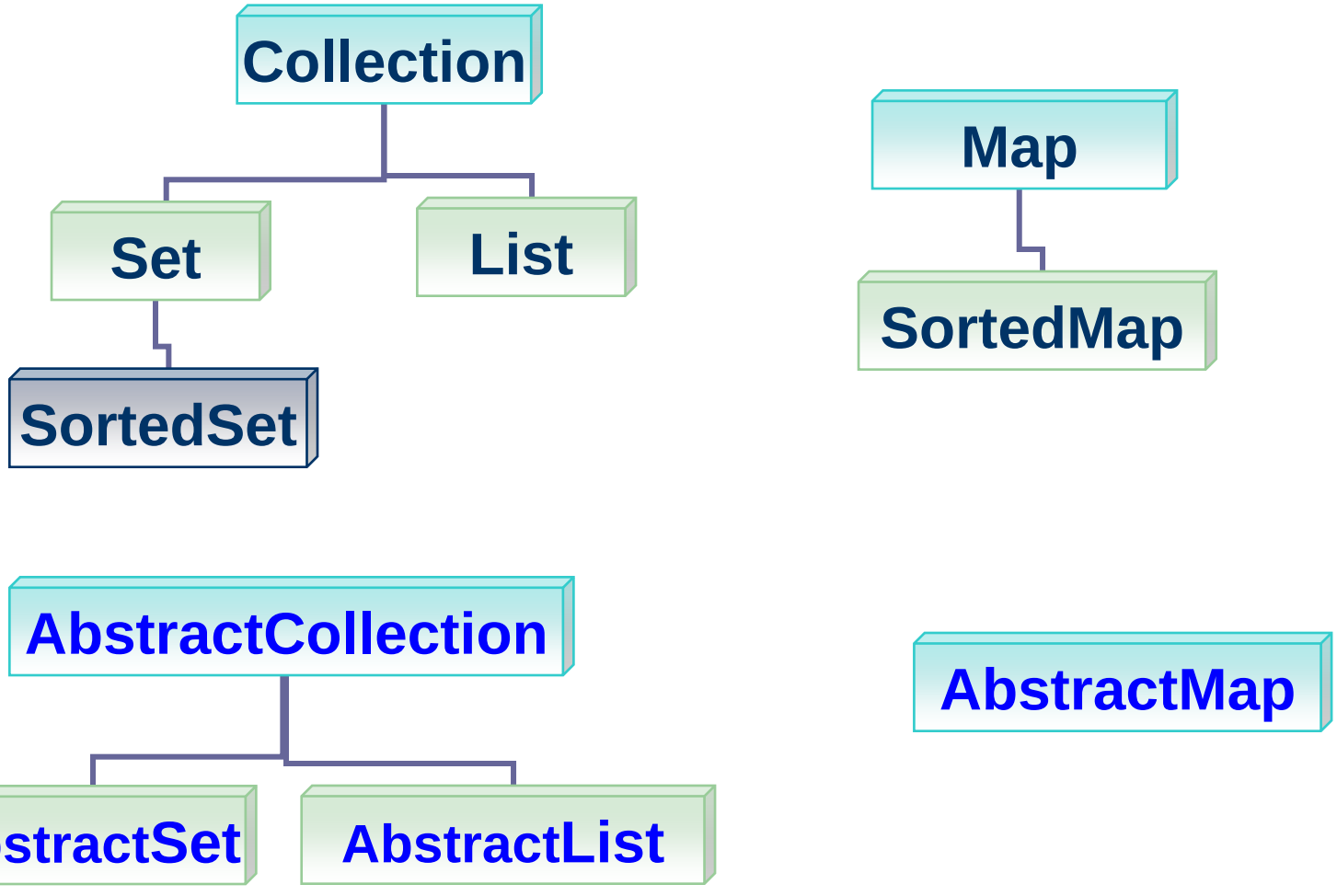
- **Interfeţe**: *tipuri abstracte de date* ce descriu colecţiile şi permit utilizarea lor independent de detaliile implementărilor.
- **Implementări**: implementări concrete ale interfeţelor ce descriu colecţii (clase); reprezintă *tipuri de date reutilizabile*.
- **Algoritmi polimorifici**: metode care efectuează diverse operaţii utile(căutare, sortare) definite pentru obiecte ce implementează interfeţele ce descriu colecţii. Aceşti algoritmi se numesc şi polimorfici deoarece pot fi folosiţi pe implementări diferite ale unei colecţii, reprezentând elementul de *funcţionalitate reutilizabilă.*

Avantaje:

- Reducerea efortului de programare
- Creşterea vitezei şi calităţii programului

Interfeţe ce descriu colecţii

- Collection - modelează o colecţie la nivelul cel mai general, descriind un grup de obiecte (elementele sale).
- Map - descrie structuri de date de tip cheie – valoare, asociază fiecarui element o cheie unică, după care poate fi regăsit.

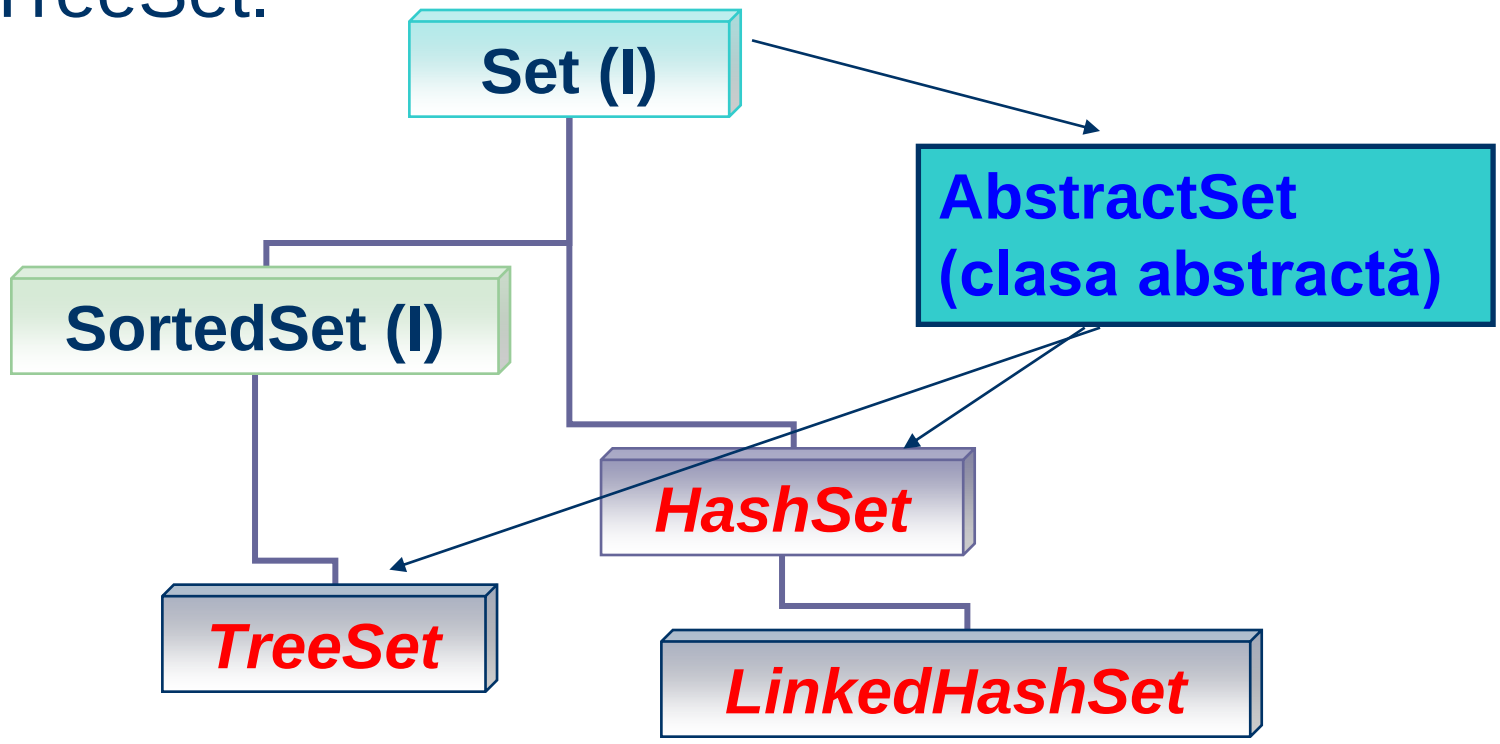


Collection

```
public interface Collection {
    // Metode cu caracter general
    int size();
    boolean isEmpty();
    void clear();
    Iterator iterator();
    // Operatii la nivel de element
    boolean contains(Object element);
    boolean add(Object element);
    boolean remove(Object element);
    // Operatii la nivel de colectie
    boolean containsAll(Collection c);
    boolean addAll(Collection c);
    boolean removeAll(Collection c);
    boolean retainAll(Collection c);
    // Metode de conversie in vector
    Object[ ] toArray();
    …
    }
```

Set

- Mulţime în sens matematic.
- O mulţime nu poate avea elemente duplicate: nu poate conţine două obiecte o1 şi o2 cu proprietatea o1.equals(o2).
- Implementări: HashSet, LinkedHashSet şi TreeSet.



SortedSet

- mulţime cu elemente sortate.
- ordonarea elementelor se face conform ordinii lor naturale, sau conform cu ordinea dată de un comparator specificat la crearea colecţiei
- ordinea este menţinută automat la orice operaţie efectuată asupra mulţimii.
- pentru orice două obiecte o1, o2 ale colecţiei, apelul o1.compareTo(o2) (sau comparator.compare(o1, o2), dacă este folosit un comparator) trebuie să fie valid şi să nu provoace excepţii

SortedSet

```
public interface SortedSet extends Set {
    // Subliste
    SortedSet subSet(Object fromElement,
                     Object toElement);
    SortedSet headSet(Object toElement);
    SortedSet tailSet(Object fromElement);
    // Capete
    Object first();
    Object last();
    Comparator comparator();
}
```

Implementare: **TreeSet – help**

HashSet, LinkedHashSet

HashSet: **tabelă de dispersie** (hash table);

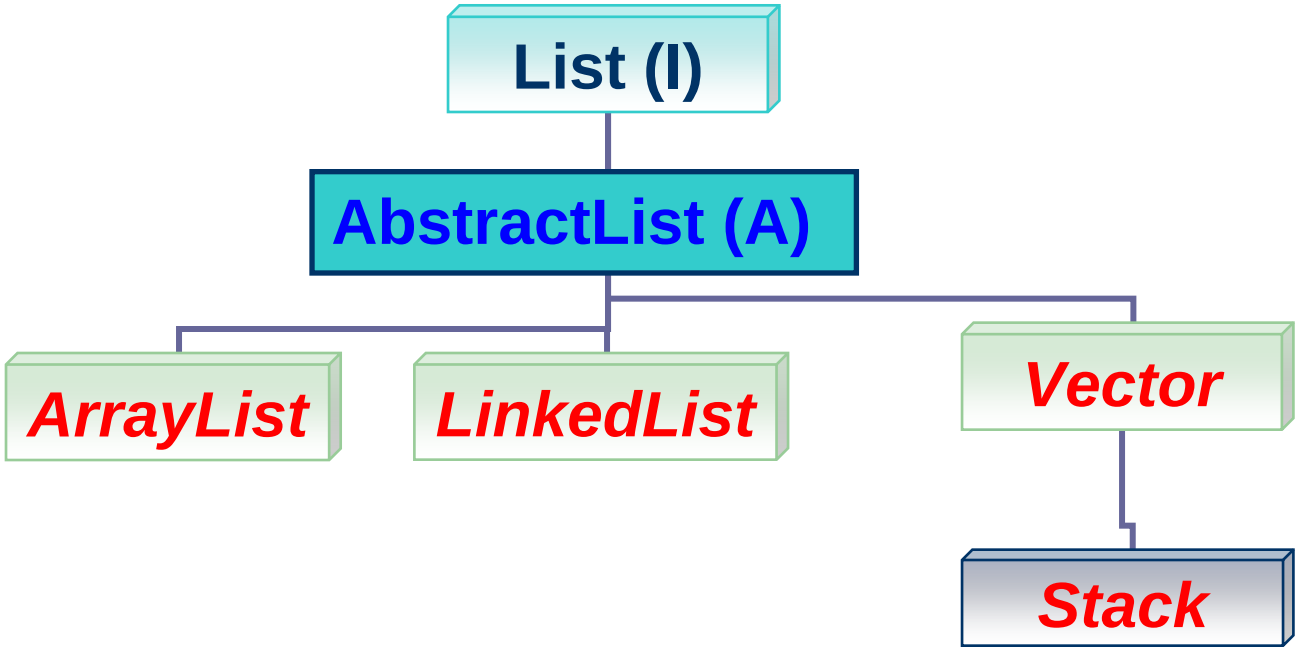
- implementarea cea mai performantă
- calculează codul de dispersie al elementelor pe baza metodei hashCode, definită în clasa Object
- nu avem garanţii asupra **ordinii** de parcurgere.
- doi iteratori **diferiţi** pot parcurge elementele mulţimii în ordine **diferită**.

LinkedHashSet : tabelă de dispersie.

Diferenţa faţă de HashSet - menţine o listă dublu-înlănţuită peste toate elementele sale. => elementele rămân în **ordinea** în care au fost inserate.

List

- liste de elemente indexate.
- pot contine duplicate şi permit un control precis asupra poziţiei unui element prin intermediul indexului acelui element.
- există metode pentru acces poziţional, căutare şi iterare avansată.
- Implementări: ArrayList, LinkedList, Vector, Stack.



List (2)

```
public interface List extends Collection {
    // Acces pozitional
    Object get(int index);
    Object set(int index, Object element);
    void add(int index, Object element);
    Object remove(int index);
    abstract boolean addAll(int index, Collection c);
    // Cautare
    int indexOf(Object o);
    int lastIndexOf(Object o);
    // Iterare
    ListIterator listIterator();
    ListIterator listIterator(int index);
    // Extragere sublista
    List subList(int from, int to);
}
```

Map

- structuri de tip: cheie – element, ce asociază fiecarui element o cheie unică, după care poate fi regăsit.
- nu pot conţine chei duplicate şi fiecare cheie este asociată unui singur element.

```
public interface Map {
    // Metode cu caracter general
    int size();
    boolean isEmpty();
    void clear();
    // Operatii la nivel de element
    Object put(Object key, Object value);
    Object get(Object key);
    Object remove(Object key);
    boolean containsKey(Object key);
    boolean containsValue(Object value);
```

Map(2)

```
// Operatii la nivel de multime
void putAll(Map t);
// Vizualizari ale colectiei
public Set keySet();
public Collection values();
public Set entrySet();
// Interfata pentru manipularea unei inregistrari
public interface Entry {
    Object getKey();
    Object getValue();
    Object setValue(Object value);
 }
}
```

Implementări: **HashMap, TreeMap, LinkedHashMap -** şi **Hashtable.**

SortedMap

- Mulţimea cheilor este sortată conform ordinii naturale sau unui comparator.

```
public interface SortedMap extends Map {
    // Extragerea de subtabele
    SortedMap subMap(Object fromKey, Object
      toKey);
    SortedMap headMap(Object toKey);
    SortedMap tailMap(Object fromKey);
    // Capete
    Object first();
    Object last();
    // Comparatorul folosit pentru ordonare
    Comparator comparator();
}
```

- Implementare: **TreeMap - help**

Interfaţa Map.Entry

- desemnează o pereche (cheie, valoare) din map.
- Metode:
  - **getKey**: întoarce cheia
  - **getValue**: întoarce valoarea
  - **setValue**: permite stabilirea valorii asociată cu această cheie

Implementări ale colecţiilor

Interfaţa	Clasa
Set	**HashSet, *LinkedHashSet***
SortedSet	**TreeSet**
List	**ArrayList, LinkedList, Vector, *Stack***
Map	**HashMap, Hashtable, LinkedHashMap**
SortedMap	**TreeMap**

- Organizare ierarhică

AbstractCollection - AbstractSet, AbstractList - HashSet, TreeSet... Vector-Stack

AbstractMap - HashMap, TreeMap, HashTable

In vechea ierarhie:

Dictionary - Hashtable – Properties

- se observă existenţa unor clase care oferă aceeaşi funcţionalite, cum ar fi ArrayList şi Vector, HashMap şi Hashtable.

Caracteristici comune

- permit elementul null
- sunt serializabile
- au definită metoda clone
- au definită metoda toString
- permit crearea de iteratori
- au atât constructor fără argumente cât şi un constructor care acceptă ca argument o altă colecţie
- exceptând clasele din arhitectura veche, nu sunt sincronizate.

Folosirea eficientă a colecţiilor

- ArrayList sau LinkedList ?

```
import java . util .*;
public class TestEficienta {
    final static int N = 100000;
    public static void testAdd ( List lst) {
       long t1 = System . currentTimeMillis ();
       for (int i=0; i < N; i++)
         lst.add (new Integer (i));
       long t2 = System . currentTimeMillis ();
       System . out. println ("Add: " + (t2 - t1));
}
public static void testGet ( List lst) {
    long t1 = System . currentTimeMillis ();
    for (int i=0; i < N; i++)
       lst.get (i);
    long t2 = System . currentTimeMillis ();
    System . out. println ("Get: " + (t2 - t1));
}
```

ArrayList sau LinkedList ?

```
public static void testRemove ( List lst ) {
      long t1 = System . currentTimeMillis ();
      for (int i=0; i < N; i++)
         lst. remove (0);
      long t2 = System . currentTimeMillis ();
      System . out. println (" Remove : " + (t2 - t1));
}
public static void main ( String args []) {
      System . out. println (" ArrayList ");
      List lst1 = new ArrayList ();
      testAdd ( lst1 );
      testGet ( lst1 );
      testRemove ( lst1 );
      System . out. println (" LinkedList ");
      List lst2 = new LinkedList ();
      testAdd ( lst2 );
      testGet ( lst2 );
      testRemove ( lst2 );
 }
}
```

ArrayList sau LinkedList ?

	ArrayList	LinkedList
add	0.12	0.14
get	0.01	87.45
remove	12.05	0.01

Concluzii:

- există diferenţe substanţiale în reprezentarea şi comportamentul diferitelor implementări
- alegerea unei anumite clase pentru reprezentarea unei mulţimi de elemente trebuie să se facă în funcţie de natura problemei ce trebuie rezolvată.

Algoritmi polimorfici

- Metode definite în clasa Collections: căutare, sortare, etc.

Caracteristici comune:

- sunt metode de clasă (statice);
- au un singur argument de tip colecţie;
- apelul lor general va fi de forma:

Collections.algoritm(colectie,[argumente]);

- majoritatea operează pe liste dar şi pe colecţii arbitrare.

Exemple de algoritmi

- sort
- shuffle
- binarySearch
- reverse
- fill
- copy
- min
- max
- swap
- enumeration
- unmodifiableTipColectie
- synchronizedTipColectie

Tipuri generice

- **Tipizarea elementelor unei colecţii:**
  TipColectie < TipDeDate >

```
// Inainte de 1.5
    ArrayList list = new ArrayList();
    list.add(new Integer(123));
    int val = ((Integer)list.get(0)).intValue();
// Dupa 1.5, folosind tipuri generice
    ArrayList<Integer> list = new ArrayList<Integer>();
    list.add(new Integer(123));
    int val = list.get(0).intValue();
// Dupa 1.5, folosind si autoboxing
    ArrayList<Integer> list = new ArrayList<Integer>();
    list.add(123);
    int val = list.get(0);
```

- **Avantaje**: simplitate, control (eroare la compilare vs. ClassCastException)

Iteratori şi enumerări

- Parcurgerea secvenţială a unei colecţii, indiferent dacă este indexată sau nu.
- Toate clasele care implementează colecţii au metode ce returnează o enumerare sau un iterator pentru parcurgerea elementelor lor.
- Deoarece funcţionalitatea interfeţei Enumeration se regăseşte în Iterator, aceasta din urmă este preferată în noile implementări ale colecţiilor.

Enumeration:

```
// Parcurgerea elementelor unui Vector v
Enumeration e = v.elements();
while (e.hasMoreElements()) {
    System.out.println(e.nextElement());
}
// sau, varianta mai concisa
for(Enumeration e=v.elements(); e.hasMoreElements();){
    System.out.println(e.nextElement());
}
```

Iterator, ListIterator

- **Iterator:** hasNext, next, remove

```
// Parcurgerea elementelor unui vector si eliminarea
    elementelor nule
for (Iterator it = v.iterator(); it.hasNext();) {
    Object obj = it.next();
    if (obj == null)   it.remove();
}
```

- **ListIterator**: hasNext, hasPrevious, next, previous, remove, add, set

```
// Parcurgerea elementelor unui vector si inlocuirea
    elementelor nule cu 0
for (ListIterator it = v.listIterator(); it.hasNext();) {
    Object obj = it.next();
    if (obj == null)    it.set(new Integer(0));
}
```

Folosirea unui iterator

```
import java . util .*;
class TestIterator {
    public static void main ( String args []) {
            ArrayList a = new ArrayList ();
            // Adaugam numerele de la 1 la 10
            for (int i=1; i <=10; i++)   a.add(new Integer (i));
            // Amestecam elementele colectiei
            Collections . shuffle (a);
            System . out. println (" Vectorul amestecat : " + a);
            // Parcurgem vectorul
            for ( ListIterator it=a. listIterator (); it. hasNext (); ) {
               Integer x = ( Integer ) it. next ();
               // Daca elementul curent este par , il facem 0
               if (x % 2 == 0)  it. set ( 0 );
            }
            System . out. print (" Rezultat : " + a);
    }
}
```

Varianta simplificată (1.5)

Atenţie!

- Deoarece colecţiile sunt construite peste tipul de date Object, metodele de tip *next* sau *prev* ale iteratorilor vor returna tipul Object, fiind responsabilitatea noastră de a face conversie (cast) la alte tipuri de date, dacă este cazul.

```
ArrayList<Integer> list=new ArrayList<Integer>();
for (Iterator i = list.iterator(); i.hasNext();) {
    Integer val=(Integer) i.next();
    // Proceseaza val
    ...
}
```

sau:

```
ArrayList<Integer> list=new ArrayList<Integer>();
for (Integer val : list) {
    // Proceseaza val
    ...
}
```

Exemple

```
Collection<String> c = new ArrayList<String>();
c.add("Test");
c.add(2); // EROARE compilare!
Iterator<String> it = c.iterator();
while (it.hasNext()) {
   String s = it.next();
}
```

- O **iterare** obişnuită pe un map *students* se va face în felul următor:

```
for (Map.Entry<String,Student> entry:
   students.entrySet())
   System.out.println("Media studentului “+
   entry.getKey()+“este“ +entry.getValue().getAverage());
```

- bucla for-each⇔ iteratorul mulţimii de perechi întoarse de entrySet.

Exemplu

1. Să se definească o clasă “MyArray" pentru vectori de obiecte care nu se extind dinamic şi o clasă iterator pe acest vector care să implementeze interfaţa "Enumeration".

Program pentru crearea unui obiect “MyArray" prin adăugări succesive de şiruri şi afişarea lui folosind un obiect iterator.

Clasa “MyArray" are un constructor cu argument întreg (dimensiune vector) şi metodele:

- add(Object): adăugare obiect la sfârşit
- elements(): creare iterator pentru acest vector
- get(int): obiect dintr-o poziţie data
- size(): dimensiune vector
- toString: şir cu elementele din vector.

Rezolvare

```
import java.util.*;

class MyArray {

  private Object v[];
  private int n;

  public MyArray (int nmax) {
        v= new Object[nmax];
        n=0;
  }

  public void add (Object e) {
    if ( n < v.length)
      v[n++]=e;
  }
```

Rezolvare

```
public Enumeration elements() {
     return new ArrayEnum(this);
  }

  public String toString () {
    String s="";
    for (int i=0;i<n;i++)
      s=s+v[i]+" ";
    return s;
  }

  public int size() {
    return n;
  }

  public Object get(int i) {
     return v[i];
  }
}
```

Rezolvare

```
// clasa iterator pentru vectori
class ArrayEnum implements Enumeration {
  private int i;
  private MyArray a;

 ArrayEnum (MyArray a) {
    this.a =a;
    i=0;
  }

 public Object nextElement() {
    return a.get(i++);
  }

  public boolean hasMoreElements() {
    return i < a.size();
  }
}
```

Rezolvare

```
class ArrayDemo {
   // afisare prin enumerare
  public static void main ( String av[]) {
    MyArray a = new MyArray(10);
    for (int i=1;i<11;i++)
      a.add(""+i);
    Enumeration it = a.elements();
    while (it.hasMoreElements())
       System.out.print (it.nextElement()+",");
    System.out.println();
    System.out.println(a);
   }
}
```